VAIDSYS TECHNOLOGIES

INTERNSHIP PROJECT REPORT

Adaptive Cache Controller with Workload-Aware Cache Management

Submitted By

Lakshmi Narendra Bokka

Internship Domain

VLSI Design and Verification

Submitted To

Vaidsys Technologies

June 30,2026

Contents

Chapter 1

Introduction

Modern processors face the memory wall problem due to the large speed gap between processor and memory. Cache memory is introduced to reduce memory access latency and improve performance. However, cache efficiency depends heavily on the replacement policy used when the cache becomes full.

Traditional replacement policies such as FIFO and LRU are effective only for certain workload types. This project proposes an adaptive cache controller capable of dynamically selecting between FIFO and LRU based on observed memory access patterns.

Chapter 2

Background Theory

2.1 Memory Hierarchy

A computer system consists of multiple levels of memory organized in a hierarchical structure. The hierarchy is designed to provide a balance between speed, cost, and storage capacity. Registers are the fastest memory elements but provide very limited storage. Main memory offers larger storage but operates at a significantly lower speed. Secondary storage devices such as SSDs and hard disks provide massive storage capacity but have the highest access latency.

The memory hierarchy is organized as follows:

1. CPU Registers
2. L1 Cache
3. L2 Cache
4. L3 Cache
5. Main Memory (RAM)
6. Secondary Storage

The cache memory serves as an intermediary between the processor and main memory. By storing frequently accessed data closer to the processor, cache memory significantly reduces average memory access time.

2.2 The Memory Wall Problem

Over the past decades, processor speeds have increased at a much faster rate than memory speeds. While CPU frequencies have improved dramatically, memory latency has not

improved proportionally. This disparity creates a bottleneck known as the **Memory Wall**.

When the processor waits for data from memory, valuable clock cycles are wasted. As applications become more data-intensive, memory latency increasingly limits overall system performance.

The memory wall problem motivates the use of sophisticated cache architectures and intelligent cache management techniques.

2.3 Cache Memory

Cache memory is a small, high-speed memory located between the CPU and main memory. Its primary purpose is to reduce memory access latency.

Cache memory exploits two important principles:

- **Temporal Locality**: Recently accessed data is likely to be accessed again.
- **Spatial Locality**: Data located near recently accessed data is likely to be accessed soon.

When requested data is found in the cache, a cache hit occurs. Otherwise, a cache miss occurs and data must be fetched from main memory.

2.4 Cache Mapping Techniques

There are three primary cache mapping techniques:

2.4.1 Direct Mapped Cache

Each memory block maps to exactly one cache location. This approach is simple and fast but suffers from high conflict misses.

2.4.2 Fully Associative Cache

A memory block can be placed anywhere in the cache. This provides maximum flexibility but requires expensive hardware.

2.4.3 Set Associative Cache

A compromise between direct-mapped and fully associative caches. The cache is divided into sets, and each memory block maps to a specific set but may occupy any way within that set.

The proposed design uses a 4-way set associative cache structure.

2.5 Cache Performance Metrics

Several metrics are used to evaluate cache performance:

2.5.1 Hit Rate

Hit rate represents the percentage of memory accesses that are successfully satisfied by the cache without accessing main memory.

$$HitRate = \frac{NumberofHits}{TotalAccesses} \times 100 \tag{2.1}$$

2.5.2 Miss Rate

Miss rate represents the percentage of memory accesses that are not found in the cache and therefore require access to main memory.

$$MissRate = \frac{NumberofMisses}{TotalAccesses} \times 100 \tag{2.2}$$

2.5.3 Average Memory Access Time

Average Memory Access Time (AMAT) is given by:

[AMAT = Hit Time + Miss Rate $\times Miss\ Penalty$]

Lower AMAT indicates better cache performance.

2.6 Cache Replacement Policies

When the cache becomes full, one cache line must be replaced to accommodate new data.

2.6.1 First-In First-Out (FIFO)

FIFO replaces the cache line that has been present in the cache for the longest duration.

Advantages:

- Simple implementation
- Low hardware overhead
- Suitable for streaming workloads

Disadvantages:

- Does not consider access frequency
- May evict frequently used data

2.6.2 Least Recently Used (LRU)

LRU replaces the cache line that has not been accessed for the longest period.

Advantages:

- Exploits temporal locality
- Higher hit rates for repeated workloads
- Widely used in modern processors

Disadvantages:

- Requires additional hardware
- Higher implementation complexity

2.7 Need for Adaptive Cache Management

Different applications exhibit different memory access patterns.

Examples include:

- Video streaming applications
- Machine learning workloads
- Database systems
- Embedded control systems

A single replacement policy cannot provide optimal performance for all workloads.

An adaptive cache controller addresses this limitation by dynamically selecting the most appropriate replacement policy according to observed workload behavior.

The adaptive strategy improves cache efficiency while maintaining low hardware overhead.

Chapter 3

Objectives

- Design a 4-way set associative cache.
- Implement FIFO replacement policy.
- Implement LRU replacement policy.
- Develop workload detection logic.
- Dynamically switch between FIFO and LRU.
- Measure cache performance using hit and miss counters.
- Verify functionality using Vivado simulation.
- Analyze area, timing and power after synthesis.

Chapter 4

System Architecture

The proposed design consists of the following modules:

1. Main Memory
2. Adaptive Selector
3. Cache Controller
4. FIFO Controller
5. LRU Controller
6. Cache Statistics

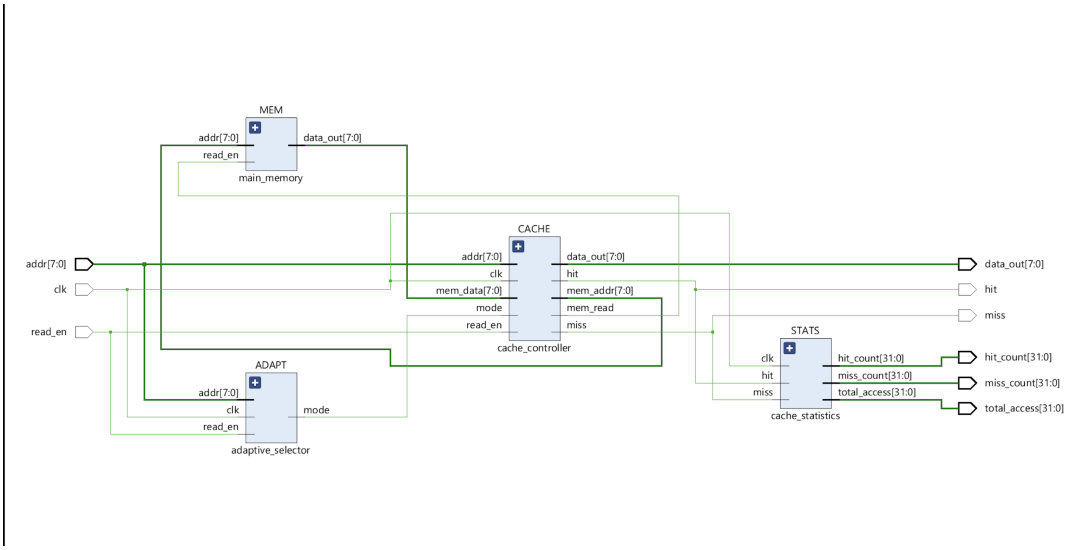

Figure 4.1: Top Level Architecture

Chapter 5

Main Memory Module

5.1 Overview

The Main Memory module represents the lower level of the memory hierarchy and serves as the source of data whenever a cache miss occurs. In practical computer systems, main memory is implemented using DRAM technology and has significantly larger storage capacity than cache memory.

In this project, a simplified 256-byte memory model is implemented using a Verilog array. Each memory location is initialized with its corresponding address value for ease of verification.

5.2 Functionality

The module receives an address and a read enable signal from the cache controller. Whenever a cache miss occurs, the cache controller forwards the requested address to the main memory.

The memory then returns the corresponding data value through the data output bus.

5.3 Inputs and Outputs

Signal	Direction	Description
read_en	Input	Enables memory read operation
addr[7:0]	Input	Memory address
data_out[7:0]	Output	Requested data

Table 5.1: Main Memory Interface

5.4 Working Principle

When read enable is asserted, the memory accesses the location specified by the address bus and returns the stored value. If read enable is deasserted, the output remains zero.

The memory contents are initialized during simulation using a loop that stores the address value into each location.

5.5 Role in Adaptive Cache System

The main memory acts as the backup storage for the cache. Whenever the cache cannot satisfy a request, the required data is fetched from main memory and inserted into the cache.

Chapter 6

Adaptive Selector Module

6.1 Overview

The Adaptive Selector is the most important component of the proposed architecture. Its purpose is to analyze memory access behavior and dynamically determine the most suitable cache replacement policy.

Traditional cache systems use a fixed replacement policy throughout execution. However, different workloads exhibit different memory access characteristics. The adaptive selector overcomes this limitation by monitoring access patterns and selecting either FIFO or LRU replacement.

6.2 Workload Analysis

Two workload characteristics are analyzed:

1. Streaming Access Pattern
2. Repeated Access Pattern

Streaming workloads access consecutive memory locations, whereas repeated workloads repeatedly access previously used addresses.

6.3 History Buffer

The module stores the most recent four addresses in a history buffer.

$$History = \{H_0, H_1, H_2, H_3\}$$

These entries are continuously updated whenever a new memory request arrives.

6.4 Streaming Detection

Streaming accesses are detected when the current address is close to the previously accessed address.

Examples include:

$$20 \rightarrow 24 \rightarrow 28 \rightarrow 32$$

Such patterns indicate sequential memory traversal.

6.5 Repeat Detection

Repeated accesses are detected when the current address matches any address stored in the history buffer.

Example:

$$20 \rightarrow 24 \rightarrow 20 \rightarrow 24$$

This pattern indicates temporal locality.

6.6 Mode Selection

The adaptive selector compares streaming and repeat counts.

- Repeat Count > Stream Count → LRU
- Stream Count > Repeat Count → FIFO

This dynamic selection enables workload-aware cache management.

Chapter 7

Cache Controller

The cache controller implements a 4-way set associative cache.

Features:

- Four sets
- Four ways per set
- Valid bits
- Tag memory
- Data memory
- Hit detection
- Miss handling
- Adaptive replacement

On a cache hit, data is returned immediately. On a miss, data is fetched from main memory and inserted according to FIFO or LRU replacement.

Chapter 8

FIFO Controller

8.1 Overview

FIFO stands for First-In First-Out. It is one of the simplest cache replacement policies and is widely used due to its low implementation complexity.

The FIFO controller maintains information about the order in which cache blocks were inserted.

8.2 Working Principle

Whenever the cache becomes full and a miss occurs, the block that entered the cache earliest is selected for replacement.

For example:

$$A \rightarrow B \rightarrow C \rightarrow D$$

If another block arrives after the cache is full, block A is removed first.

8.3 Implementation

A circular pointer is maintained for every cache set.

The pointer indicates the next cache way that will be replaced.

After replacement, the pointer increments automatically.

8.4 Advantages

- Simple hardware implementation
- Low area overhead

- Suitable for streaming workloads
- Fast victim selection

8.5 Limitations

- Ignores access frequency
- Frequently used blocks may be removed
- Lower performance for repeated-access workloads

Chapter 9

LRU Controller

9.1 Overview

LRU stands for Least Recently Used. It is one of the most commonly used cache replacement policies because it effectively exploits temporal locality.

The basic assumption behind LRU is that recently accessed data is likely to be accessed again in the near future.

9.2 Working Principle

Each cache line is assigned an age value.

Whenever a cache line is accessed:

- Its age becomes zero.
- Ages of other cache lines increase.

The cache line with the highest age is selected for replacement.

9.3 Example

Assume four cache lines:

$$A, B, C, D$$

If A is accessed, its age becomes zero.
If B is accessed next, A becomes older and B becomes most recently used.
Eventually the oldest block is identified as the victim.

9.4 Advantages

- High hit rate
- Exploits temporal locality
- Better performance for repeated workloads
- Widely used in processor caches

9.5 Limitations

- Higher hardware complexity
- Additional storage for age information
- Increased update overhead

Chapter 10

Cache Statistics Module

10.1 Overview

Performance evaluation is an essential part of cache design. The Cache Statistics module measures the effectiveness of the adaptive cache architecture.

10.2 Metrics Collected

The module maintains three counters:

1. Hit Counter
2. Miss Counter
3. Total Access Counter

10.3 Hit Counter

The hit counter increments whenever the requested data is found inside the cache.

A high hit count indicates efficient cache utilization.

10.4 Miss Counter

The miss counter increments whenever the requested data is absent from the cache.

Misses require data retrieval from main memory and increase memory access latency.

10.5 Total Access Counter

The total access counter tracks the overall number of memory requests processed by the system.

10.6 Performance Evaluation

Using these counters, hit rate and miss rate can be calculated as:

$$Hit\ Rate = \frac{Hits}{Total\ Accesses} \times 100$$

$$Miss\ Rate = \frac{Misses}{Total\ Accesses} \times 100$$

These metrics provide a quantitative measure of cache efficiency.

Chapter 11

Design Methodology

The design process followed a modular hardware design approach.

11.1 Step 1: Cache Organization

A 4-way set associative cache was selected to balance hardware complexity and cache performance.

The cache consists of:

- 4 sets
- 4 ways per set
- Valid bit storage
- Tag storage
- Data storage

11.2 Step 2: Workload Detection

The adaptive selector continuously monitors recent memory accesses.

A history buffer stores the last four accessed addresses. Using this information, the module computes:

- Stream count
- Repeat count

These counters are used to classify workload behavior.

11.3 Step 3: Policy Selection

The adaptive selector compares stream count and repeat count.

- Repeat count greater than stream count → LRU mode
- Stream count greater than repeat count → FIFO mode

11.4 Step 4: Cache Access

For each memory request:

1. Extract tag and index.
2. Search all four ways.
3. Generate hit or miss signal.
4. Return data on hit.
5. Initiate replacement on miss.

11.5 Step 5: Performance Evaluation

The design was evaluated using:

- Functional simulation
- Waveform verification
- Resource utilization analysis
- Timing analysis
- Power analysis

Chapter 12

Simulation Results

Three workloads were tested.

12.1 Repeated Access Workload

Address Pattern:

`20,24,20,24,20,24,36,20`

Results:

- Hits = 13
- Misses = 3
- Total Accesses = 16
- Hit Rate = 81.25%

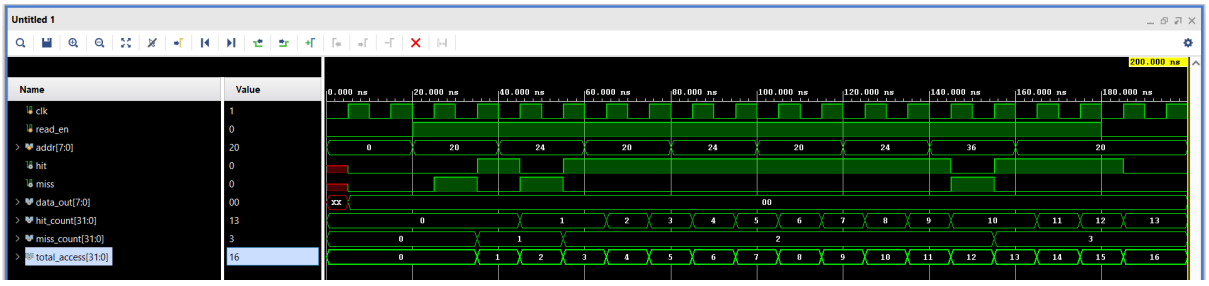

Figure 12.1: Repeated Access Workload

The adaptive selector remained in LRU mode because repeated accesses dominated.

12.2 Streaming Workload

Address Pattern:

`4,8,12,16,20,24,28,32`

Results:

- Hits = 8
- Misses = 8
- Total Accesses = 16
- Hit Rate = 50%

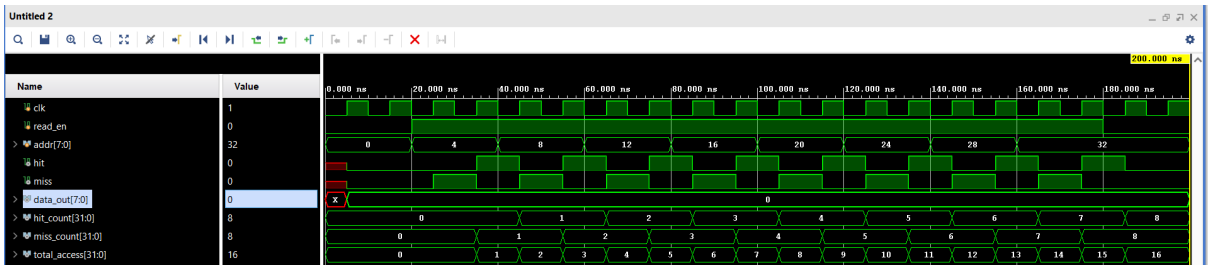

Figure 12.2: Streaming Workload

The adaptive selector frequently switched to FIFO mode because sequential accesses dominated.

12.3 Mixed Workload

Address Pattern:

`20,24,28,20,24,32,36,20`

Results:

- Hits = 10
- Misses = 6
- Total Accesses = 16
- Hit Rate = 62.5%

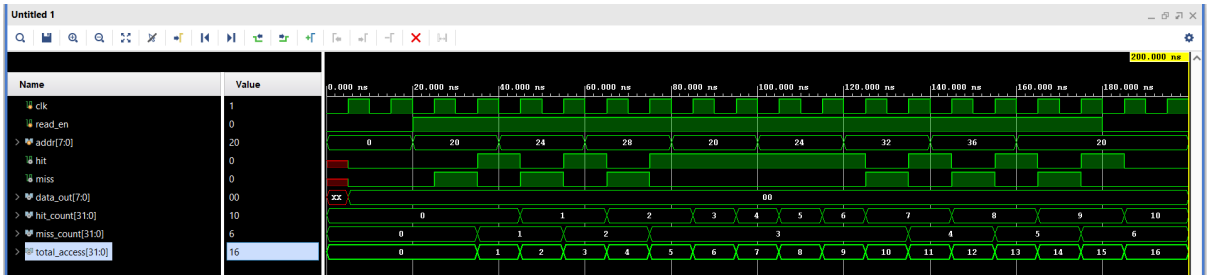

Figure 12.3: Mixed Workload

The selector balanced between FIFO and LRU behavior.

Chapter 13

Synthesis Results

13.1 FPGA Resource Utilization

Resource	Used	Available	Utilization
LUT	232	53200	0.44%
FF	424	106400	0.40%
BRAM	0.5	140	0.36%
BUFG	1	32	3.13%

Table 13.1: Resource Utilization

13.2 Timing Results

Parameter	Value
WNS	3.433 ns
TNS	0 ns
WHS	0.185 ns
Failing Endpoints	0

Table 13.2: Timing Summary

All timing constraints are satisfied.

13.3 Power Results

Parameter	Value
Total On-Chip Power	0.129 W
Dynamic Power	0.024 W
Static Power	0.105 W
Junction Temperature	26.5 C

Table 13.3: Power Summary

Chapter 14

Advantages

The proposed Adaptive Cache Controller offers several benefits compared to traditional fixed-policy cache systems. By dynamically selecting between FIFO and LRU replacement policies, the cache can adapt to different workload characteristics and improve overall performance.

The design effectively handles both streaming and repetitive access patterns. FIFO replacement is suitable for sequential accesses, while LRU performs better for workloads exhibiting temporal locality. The adaptive selector automatically chooses the appropriate policy, reducing cache misses and improving hit rates.

Another important advantage is the low hardware overhead. The adaptive mechanism requires only simple counters and history registers, making the implementation resource-efficient. FPGA synthesis results confirm that the design occupies only a small percentage of available hardware resources while maintaining low power consumption.

The modular architecture also improves scalability and makes the design suitable for future enhancements such as larger caches, write policies, and processor integration.

Chapter 15

Future Scope

The current implementation focuses on adaptive replacement policy selection for read operations. Several enhancements can be incorporated in future versions of the design.

Support for write-back and write-through cache policies can be added to improve functionality. Dirty-bit management can also be implemented to reduce unnecessary memory accesses. The cache size and associativity can be increased to evaluate performance under larger workloads.

Future work may include integrating the adaptive cache controller with a RISC-V processor and evaluating its behavior using real application traces. Advanced techniques such as hardware prefetching and machine-learning-based cache prediction can also be explored to further improve cache performance and adaptability.

Chapter 16

Conclusion

This project successfully implemented an Adaptive Cache Controller using Verilog HDL. The design combines FIFO and LRU replacement policies with an adaptive selector capable of analyzing workload behavior and dynamically choosing the most suitable policy.

Functional verification using Vivado simulations demonstrated correct cache operation, hit and miss detection, replacement policy switching, and statistics collection. Multiple workload patterns, including streaming, repetitive, and mixed accesses, were tested to evaluate performance.

Synthesis and implementation results showed low FPGA resource utilization, successful timing closure, and low power consumption. Overall, the project provided valuable experience in cache architecture, RTL design, verification, and performance analysis while demonstrating the effectiveness of workload-aware cache management.

References

1. David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th Edition, Morgan Kaufmann Publishers.

2. John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th Edition, Morgan Kaufmann Publishers.

3. Alan Jay Smith, "Cache Memories," *ACM Computing Surveys*, Vol. 14, No. 3, pp. 473–530, 1982.

4. Norman P. Jouppi, "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers," Proceedings of the 17th Annual International Symposium on Computer Architecture, 1990.

5. Xilinx Inc., *Vivado Design Suite User Guide*, Available at: https://www.xilinx.com

6. Xilinx Inc., *Vivado Design Suite User Guide: Synthesis*, UG901.

7. Xilinx Inc., *Vivado Design Suite User Guide: Implementation*, UG904.

8. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson Education.

9. Douglas J. Smith, *HDL Chip Design*, Doone Publications.

10. RISC-V International, *The RISC-V Instruction Set Manual*, Available at: https://riscv.org

11. FPGA implementation reports generated using Xilinx Vivado Design Suite.

12. Simulation waveforms and verification results obtained from custom Verilog testbenches developed during the internship project.